TECHNICAL UNIVERSITY OF MOLDOVA
Faculty of Computers, Informatics, and Microelectronics
Department of Software Engineering
Software Engineering Study Programme

REPORT ON LABORATORY WORK NO. 3

OPERATING SYSTEMS

The Command Line: Shell, Pipes and Text Filters

Done by: st. gr. FAF-241, Lucian-Adrian Gavril

Checked by: univ. assist., Reițman Patricia

Chișinău, 2026

TABLE OF CONTENTS

INTRODUCTION 3

1 MOVE FAST, FIND HELP 4

1.1 Tab completion and history search 4

1.2 Built-in or external, and where the shell looks 4

1.3 Answers to the Observe questions 5

2 WHAT THE SHELL DOES TO YOUR LINE 6

2.1 The play directory 6

2.2 Predictions against real output 6

2.3 Answers to the Observe questions 8

3 REDIRECTION, CHAINING AND JOBS 10

3.1 Answers to the Observe questions 11

4 ANSWERING QUESTIONS WITH ONE-LINERS 12

4.1 Q1. How many requests failed? 12

4.2 Q2. How many different pages, and which? 12

4.3 Q3. How many requests did each page get? 12

4.4 Q4. Which user or users have the most failed requests? 13

4.5 Q5. How many requests did user3 make, and how many failed? 13

4.6 Q6. The last 3 failed requests, time and user only 13

4.7 Q7. Login shells in /etc/passwd 13

4.8 Q8. Why do ls /etc | wc -l and ls -l /etc | wc -l differ by one? 14

4.9 Bonus: minute ending in 0 and failed 14

5 YOUR FIRST SCRIPT 16

5.1 report.sh 16

5.2 Answers to the Observe questions 17

5.3 Bonus: running report.sh with cron 17

CONCLUSIONS 19

REFERENCES 20

INTRODUCTION

Purpose of the work. To work fluently in Bash, to predict what the shell does to a command line before it runs, to send output and errors exactly where they are wanted, and to answer real questions about a data set with one line of small filters joined by pipes. The last part turns the best one-liners into a script with argument checking and runs it automatically with cron.

Environment. Debian GNU/Linux 13 (trixie), GNU Bash 5.2.37, GNU coreutils 9.7, user `dev1`, working directory `~/os-lab3`. The data set `access.log` was generated with the exact block from the handout; `wc -l access.log` printed `500 access.log`. Every terminal figure is the real output of the command shown in it.

Method. For every command marked PREDICT the prediction was written down first and then compared with the real output (Table 2.1). Every answer to an *Observe* question is backed by a figure.

1 MOVE FAST, FIND HELP

1.1 Tab completion and history search

`ls /usr/sh` has a single match, so the first Tab already completes it in place to `/usr/share/`; the second Tab has nothing left to list. With a shorter, ambiguous prefix (`/usr/s`) the double Tab lists every candidate: `sbin/ share/ src/`. `cd /et` plus one Tab completes to `cd /etc/`, including the trailing slash, because `/etc` is the only match. After running `date`, `whoami` and `uname -r`, `Ctrl+R` followed by `da` found `date` in the history, and Enter executed it again (Figure 1.1).

```
dev1@dev1:~/os-lab3$ ls /usr/sh<Tab><Tab>  ->  ls /usr/share/   (single match: completed in place)
dev1@dev1:~/os-lab3$ ls /usr/s<Tab><Tab>
sbin/ share/ src/
dev1@dev1:~/os-lab3$ cd /et<Tab>  ->  cd /etc/
dev1@dev1:~/os-lab3$ <Ctrl+R> da  ->  (reverse-i-search)`da': date
dev1@dev1:~/os-lab3$ date
Mon Nov  2 17:26:28 EET 2026
```

Figure 1.1 – Tab completion and Ctrl+R history search

1.2 Built-in or external, and where the shell looks

```
dev1@dev1:~/os-lab3$ type cd ; type ls ; type echo ; which ls
cd is a shell builtin
ls is /usr/bin/ls
echo is a shell builtin
/usr/bin/ls
```

Figure 1.2 – Output of type and which

```
dev1@dev1:~/os-lab3$ echo $PATH
/usr/local/bin:/usr/bin:/bin:/usr/games
dev1@dev1:~/os-lab3$ echo $PATH | tr ':' '\n'
/usr/local/bin
/usr/bin
/bin
/usr/games
dev1@dev1:~/os-lab3$ echo $PATH | tr ':' '\n' | wc -l
4
```

Figure 1.3 – The directories in PATH

```
dev1@dev1:~/os-lab3$ nosuchprog ; echo "exit code: $?"
bash: nosuchprog: command not found
exit code: 127
dev1@dev1:~/os-lab3$ man ls 2>/dev/null | grep -E '^ +-t +'
       -t     sort by time, newest first; see --time
dev1@dev1:~/os-lab3$ man -k "list directory"
dir (1)              - list directory contents
ls (1)               - list directory contents
ntfsls (8)           - list directory contents on an NTFS filesystem
vdir (1)             - list directory contents
```

Figure 1.4 – Exit code of a missing command, the -t option in man ls, and man -k

1.3 Answers to the Observe questions

a) **Which of cd, ls, echo are built-ins, and why MUST cd be one?** `cd` and `echo` are shell built-ins; `ls` is an external program, `/usr/bin/ls` (Figure 1.2). The current directory is a property of each process. An external `cd` would run in a child process created with `fork()`, change the directory of that child with `chdir()`, and exit; the shell itself would stay where it was. Only the shell process can change its own directory, so `cd` must run inside it.

b) **How many directories are in your PATH, and in which one is ls?** 4 directories: `/usr/local/bin`, `/usr/bin`, `/bin`, `/usr/games` (Figure 1.3). `ls` is in `/usr/bin`; it is found there first because the directories are searched left to right.

c) **What exit code does a command that is not found return?** `127` (Figure 1.4). For comparison, a file that exists but is not executable gives `126` (Figure 5.2).

d) **Which option of ls sorts by modification time?** `-t`, "sort by time, newest first" (Figure 1.4); `ls -lt` shows the times and `ls -ltr` puts the newest file last.

2 WHAT THE SHELL DOES TO YOUR LINE

2.1 The play directory

```
dev1@dev1:~/os-lab3$ mkdir -p ~/os-lab3/play && cd ~/os-lab3/play
dev1@dev1:~/os-lab3/play$ touch a.c b.c file1.txt file2.txt notes.txt "alfa beta.txt"
dev1@dev1:~/os-lab3/play$ ls
 a.c  'alfa beta.txt'   b.c   file1.txt   file2.txt   notes.txt
```

Figure 2.1 – Files created in the play directory

2.2 Predictions against real output

Before `echo` or `ls` runs, the shell performs brace expansion, tilde and parameter expansion, command substitution, arithmetic expansion, word splitting, pathname expansion (globbing) and quote removal. The program receives only the resulting `argv[]`.

Command	My prediction	Real output	Explanation (if different)
`echo *.c`	`a.c b.c`	`a.c b.c`	Matches.
`echo file?.txt`	`file1.txt file2.txt`	`file1.txt file2.txt`	Matches.
`echo [a-b]*`	`a.c b.c`	`a.c alfa beta.txt b.c`	**Differs.** I forgot that `alfa beta.txt` also starts with `a`. The glob produces it as *one* word, but `echo` separates its arguments with spaces, so it looks like two names.
`echo *.xyz`	`*.xyz`	`*.xyz`	Matches: with no match Bash leaves the pattern unchanged (no `nullglob`).
`ls alfa beta.txt`	error	two errors: `cannot access 'alfa'`, `cannot access 'beta.txt'`	Matches, but there are two messages, one per word: `ls` received two arguments.
`ls "alfa beta.txt"`	`alfa beta.txt`	`'alfa beta.txt'`	**Differs:** GNU `ls` prints the name in quotes (see Q4 below).
`echo '$HOME' "$HOME" \$HOME`	`$HOME /home/dev1 $HOME`	`$HOME /home/dev1 $HOME`	Matches.
`echo {1..5} ; echo $((7 * 6))`	`1 2 3 4 5 and 42`	`1 2 3 4 5 and 42`	Matches.
`echo "today is $(date +%A), I am $(whoami)"`	`today is Monday, I am dev1`	`today is Monday, I am dev1`	Matches: substitution works inside double quotes.
`touch log{1..3}.txt && ls log*`	`log1.txt log2.txt log3.txt`	the same, one per line	**Differs in layout:** `ls` writes one name per line when its output is not a terminal.

Table 2.1 – Predictions compared with the real output

```
dev1@dev1:~/os-lab3/play$ echo *.c
a.c b.c
dev1@dev1:~/os-lab3/play$ echo file?.txt
file1.txt file2.txt
dev1@dev1:~/os-lab3/play$ echo [a-b]*
a.c alfa beta.txt b.c
dev1@dev1:~/os-lab3/play$ echo *.xyz
*.xyz
dev1@dev1:~/os-lab3/play$ echo {1..5} ; echo $((7 * 6))
1 2 3 4 5
42
dev1@dev1:~/os-lab3/play$ echo "today is $(date +%A), I am $(whoami)"
today is Monday, I am dev1
dev1@dev1:~/os-lab3/play$ touch log{1..3}.txt && ls log*
log1.txt
log2.txt
log3.txt
```

Figure 2.2 – Globbing, brace, arithmetic and command substitution results

```
dev1@dev1:~/os-lab3/play$ ls alfa beta.txt
ls: cannot access 'alfa': No such file or directory
ls: cannot access 'beta.txt': No such file or directory
dev1@dev1:~/os-lab3/play$ ls "alfa beta.txt"
'alfa beta.txt'
dev1@dev1:~/os-lab3/play$ ls alfa\ beta.txt
'alfa beta.txt'
dev1@dev1:~/os-lab3/play$ ls --quoting-style=literal "alfa beta.txt"
alfa beta.txt
```

Figure 2.3 – The ls error on a name with a space, and three fixes

```
dev1@dev1:~/os-lab3/play$ echo '$HOME' "$HOME" \$HOME
$HOME /home/dev1 $HOME
dev1@dev1:~/os-lab3/play$ echo '$(whoami)' "$(whoami)"
$(whoami) dev1
dev1@dev1:~/os-lab3/play$ echo "*.c" *.c
*.c a.c b.c
```

Figure 2.4 – Single quotes, double quotes and backslash

2.3 Answers to the Observe questions

a) **Prediction against output.** See Table 2.1: 7 of 10 lines matched exactly; the three differences are explained in the table.

b) **echo knows nothing about files, yet echo *.c printed file names. Who produced them?** The shell. During pathname expansion Bash reads the directory, sorts the names that match `*.c`, and replaces the pattern with them, so `echo` is started with `argv = {"echo", "a.c", "b.c"}` and simply prints its arguments. `echo "*.c"` prints `*.c` because quotes switch globbing off (Figure 2.4).

c) **Why did ls alfa beta.txt fail? Give two different ways to fix it.** The unquoted space splits the line into two words, so `ls` looks for two files, `alfa` and `beta.txt`, and neither exists. Fixes (Figure 2.3): quote the name, `ls "alfa beta.txt"` (or `'alfa beta.txt'`), or escape the space, `ls alfa\ beta.txt`. Tab completion produces the escaped form automatically.

d) **What is the difference between single and double quotes?** Inside single quotes nothing is expanded: `'$HOME'` and `'$(whoami)'` stay literal. Inside double quotes `$` expansions and command substitution still work (`"$HOME"` becomes `/home/dev1`), but word splitting and globbing do not. A backslash quotes a single character (Figure 2.4).

e) **Why does ls show 'alfa beta.txt' in quotes?** Since coreutils 8.25, `ls` uses `--quoting-style=shell-escape` by default when it writes to a terminal. It quotes names with spaces or special characters, so the printed name can be copied back into a command as one argument. With `--quoting-style=literal` the quotes disappear (Figure 2.3).

3 REDIRECTION, CHAINING AND JOBS

```
dev1@dev1:~/os-lab3/play$ cd ~/os-lab3
dev1@dev1:~/os-lab3$ ls /etc /nope > out.txt 2> err.txt
dev1@dev1:~/os-lab3$ wc -l out.txt ; cat err.txt
211 out.txt
ls: cannot access '/nope': No such file or directory
dev1@dev1:~/os-lab3$ head -3 out.txt
/etc:
ImageMagick-7
NetworkManager
dev1@dev1:~/os-lab3$ wc -l < access.log ; wc -l access.log
500
500 access.log
```

Figure 3.1 – stdout and stderr in separate files

```
dev1@dev1:~/os-lab3$ ls /etc /nope > all.txt 2>&1 ; tail -2 all.txt
xdg
xml
dev1@dev1:~/os-lab3$ grep -c nope all.txt
1
dev1@dev1:~/os-lab3$ ls /etc /nope 2>&1 > all.txt
ls: cannot access '/nope': No such file or directory
dev1@dev1:~/os-lab3$ grep -c nope all.txt
0
```

Figure 3.2 – Comparing the order of redirections

```
dev1@dev1:~/os-lab3$ echo first > notes.md ; echo second >> notes.md ; cat notes.md
first
second
dev1@dev1:~/os-lab3$ ls /nope 2> /dev/null || echo "failed with code $?"
failed with code 2
dev1@dev1:~/os-lab3$ ls /nope 2> /dev/null && echo "not printed" ; echo "; runs anyway"
; runs anyway
dev1@dev1:~/os-lab3$ mkdir -p tmp && cd tmp && echo "I am in $(pwd)" ; cd ~/os-lab3
I am in /home/dev1/os-lab3/tmp
```

Figure 3.3 – The ; && || operators

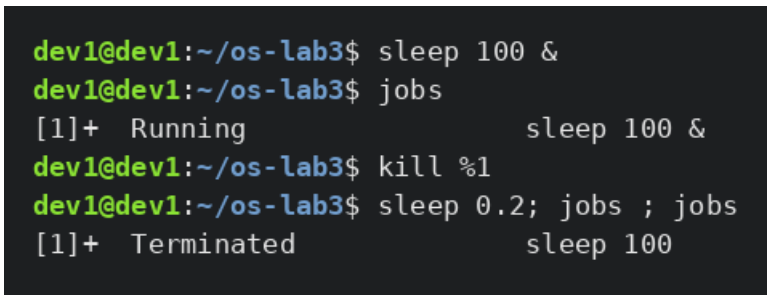


Figure 3.4 – Running a background job and stopping it

3.1 Answers to the Observe questions

a) **Which file got the error message, and which got the listing?** `out.txt` got the listing of `/etc` (stdout, file descriptor 1); `err.txt` got the one line `ls: cannot access '/nope': No such file or directory` (stderr, file descriptor 2), as Figure 3.1 shows.

b) **What is the difference between > all.txt 2>&1 and 2>&1 > all.txt?** Redirections are applied left to right, and `2>&1` means "make fd 2 a copy of fd 1 *as it is now*". In `> all.txt 2>&1`, fd 1 is first pointed at the file, then fd 2 copies it, so both streams go to `all.txt`: `grep -c nope all.txt` prints `1`. In `2>&1 > all.txt`, fd 2 copies fd 1 while fd 1 is still the terminal, and only afterwards is fd 1 moved to the file. The error therefore appears on the screen and the file contains only the listing (`grep -c` prints `0`), as Figure 3.2 shows.

c) **Why does wc -l < access.log print only a number?** With < the shell opens the file and connects it to fd 0 before `wc` starts; `wc` reads anonymous standard input and never learns a file name. With `wc -l access.log`, `wc` opens the file itself and prints the name it was given: `500` against `500 access.log` (Figure 3.1).

d) **Explain ; && ||.** ; runs the next command unconditionally; `&&` runs it only if the previous command succeeded (exit status 0); `||` runs it only if the previous command failed (non-zero status). In Figure 3.3, `ls /nope || echo` printed `failed with code 2`, while the text after `&&` was never printed.

In Figure 3.4, `sleep 100 &` starts the job in the background, so the prompt returns immediately. `jobs` shows it as `Running`; `kill %1` sends it SIGTERM, and the next `jobs` reports it as `Terminated` once and then no longer lists it.

4 ANSWERING QUESTIONS WITH ONE-LINERS

Each line of `access.log` has five space-separated fields: date, time, user, result, page. All answers are in `oneliners.sh`, one line per question, and the file runs with `bash oneliners.sh` inside `~/os-lab3` (Figure 4.9). Both self-checks from the handout pass: the Q3 counts add up to 500, and the per-user failure counts add up to the Q1 total of 100.

4.1 Q1. How many requests failed?

`grep -c FAIL access.log` gives **100**. This is consistent with the generator: a line is FAIL when $i \bmod 5 = 0$, and $500/5 = 100$.

```
dev1@dev1:~/os-lab3$ grep -c FAIL access.log
100
```

Figure 4.1 – Number of failed requests

4.2 Q2. How many different pages, and which?

`cut -d' ' -f5 access.log | sort -u | awk '{print} END{print NR " different pages"}'` gives **4 pages**: `/page0`, `/page1`, `/page2`, `/page3`.

```
dev1@dev1:~/os-lab3$ cut -d' ' -f5 access.log | sort -u | awk '{print} END{print NR " different pages"}'
/page0
/page1
/page2
/page3
4 different pages
```

Figure 4.2 – Different pages requested

4.3 Q3. How many requests did each page get?

`cut -d' ' -f5 access.log | sort | uniq -c` gives **125** for each page, $4 \times 125 = 500$. The second command in Figure 4.3 checks that the sum is 500.

```
dev1@dev1:~/os-lab3$ cut -d' ' -f5 access.log | sort | uniq -c
    125 /page0
    125 /page1
    125 /page2
    125 /page3
dev1@dev1:~/os-lab3$ cut -d' ' -f5 access.log | sort | uniq -c | awk '{s+=$1} END{print "sum:", s}'
sum: 500
```

Figure 4.3 – Requests per page and the check that they add up to 500

4.4 Q4. Which user or users have the most failed requests?

`grep FAIL access.log | cut -d' ' -f3 | sort | uniq -c | sort -rn | awk 'NR==1{max=$1} $1==max'` prints every user tied at the maximum, not a fixed number of lines: **user3 and user5, 15 failures each**. The other five users have 14 each, and the sum over users is $2 \cdot 15 + 5 \cdot 14 = 100$, equal to Q1.

```
dev1@dev1:~/os-lab3$ grep FAIL access.log | cut -d' ' -f3 | sort | uniq -c | sort -rn | awk 'NR==1{max=$
1} $1==max'
     15 user5
     15 user3
dev1@dev1:~/os-lab3$ grep FAIL access.log | cut -d' ' -f3 | sort | uniq -c | awk '{s+=$1} END{print "sum
 over users:", s}'
sum over users: 100
```

Figure 4.4 – Users with the most failed requests and the check against Q1

4.5 Q5. How many requests did user3 make, and how many failed?

`grep -w` matches `user3` as a whole word: **72 requests, 15 failed**. As a cross-check, user3 is $i \equiv 3 \pmod 7$, which gives 72 values in 1..500. A request fails for $i \equiv 0 \pmod 5$ as well, i.e. $i \equiv 10 \pmod{35}$, which gives 15 values.

```
dev1@dev1:~/os-lab3$ echo "user3: $(grep -cw user3 access.log) requests, $(grep -w user3 access.log | gr
ep -c FAIL) failed"
user3: 72 requests, 15 failed
```

Figure 4.5 – Requests and failures of user3

4.6 Q6. The last 3 failed requests, time and user only

`grep FAIL access.log | tail -n 3 | cut -d' ' -f2,3` gives `10:10 user0`, `10:15 user5`, `10:20 user3` (lines 490, 495 and 500).

```
dev1@dev1:~/os-lab3$ grep FAIL access.log | tail -n 3 | cut -d' ' -f2,3
10:10 user0
10:15 user5
10:20 user3
```

Figure 4.6 – The last three failed requests

4.7 Q7. Login shells in /etc/passwd

`cut -d: -f7 /etc/passwd | sort | uniq -c | sort -rn` gives 29 accounts with `/usr/sbin/nologin`, 3 with `/bin/false`, 3 with `/bin/bash` and 1 with `/bin/sync`. Most accounts are system services that must never log in interactively.

```
dev1@dev1:~/os-lab3$ cut -d: -f7 /etc/passwd | sort | uniq -c | sort -rn
     29 /usr/sbin/nologin
      3 /bin/false
      3 /bin/bash
      1 /bin/sync
```

Figure 4.7 – Accounts per login shell

4.8 Q8. Why do ls /etc | wc -l and ls -l /etc | wc -l differ by one?

Prediction: the long format adds one header line. **Result:** 210 against 211, and the extra line is `total 1064`, the number of 1 KiB blocks used by the listed entries. Every other line is one entry in both formats, so the difference is always exactly one.

```
dev1@dev1:~/os-lab3$ ls /etc | wc -l ; ls -l /etc | wc -l
210
211
dev1@dev1:~/os-lab3$ ls -l /etc | head -2
total 1064
drwxr-xr-x  2 root   root    4096 Oct 19 01:31 ImageMagick-7
```

Figure 4.8 – Line counts with and without -l

4.9 Bonus: minute ending in 0 and failed

`grep -E '^[^ ]+ 10:[0-5]0 [^ ]+ FAIL ' access.log` anchors the time to the second field and the result to the fourth, so a `0` in another field cannot match. It prints **50 lines**. This agrees with arithmetic: the minute is $i \bmod 60$, so it ends in 0 exactly when $i \equiv 0 \pmod{10}$, and every such i is also divisible by 5, hence FAIL. There are $500/10 = 50$ such lines.

```
dev1@dev1:~/os-lab3$ bash oneliners.sh
oneliners.sh: line 4: $'\r': command not found
--- Q1
grep: access.log
: No such file or directory
oneliners.sh: line 8: $'\r': command not found
--- Q2
/page0
/page1
/page2
/page3
4 different pages
oneliners.sh: line 12: $'\r': command not found
--- Q3
uniq: invalid option -- '
'
Try 'uniq --help' for more information.
oneliners.sh: line 16: $'\r': command not found
--- Q4
     15 user5
     15 user3
oneliners.sh: line 20: $'\r': command not found
--- Q5
user3: 72 requests, 15 failed
oneliners.sh: line 24: $'\r': command not found
--- Q6
cut: invalid field value ‘\r’
Try 'cut --help' for more information.
oneliners.sh: line 28: $'\r': command not found
--- Q7
sort: invalid option -- '
'
Try 'sort --help' for more information.
oneliners.sh: line 32: $'\r': command not found
--- Q8
210
211
head: invalid trailing option --
Try ‘head --help' for more information.
oneliners.sh: line 36: $'\r': command not found
--- Bonus
2026-10-05 10:10 user3 FAIL /page2
2026-10-05 10:20 user6 FAIL /page0
2026-10-05 10:30 user2 FAIL /page2
2026-10-05 10:40 user5 FAIL /page0
2026-10-05 10:50 user1 FAIL /page2
2026-10-05 10:00 user4 FAIL /page0
2026-10-05 10:10 user0 FAIL /page2
2026-10-05 10:20 user3 FAIL /page0
2026-10-05 10:30 user6 FAIL /page2
2026-10-05 10:40 user2 FAIL /page0
2026-10-05 10:50 user5 FAIL /page2
2026-10-05 10:00 user1 FAIL /page0
2026-10-05 10:10 user4 FAIL /page2
2026-10-05 10:20 user0 FAIL /page0
2026-10-05 10:30 user3 FAIL /page2
2026-10-05 10:40 user6 FAIL /page0
2026-10-05 10:50 user2 FAIL /page2
2026-10-05 10:00 user5 FAIL /page0
2026-10-05 10:10 user1 FAIL /page2
2026-10-05 10:20 user4 FAIL /page0
2026-10-05 10:30 user0 FAIL /page2
2026-10-05 10:40 user3 FAIL /page0
2026-10-05 10:50 user6 FAIL /page2
2026-10-05 10:00 user2 FAIL /page0
2026-10-05 10:10 user5 FAIL /page2
2026-10-05 10:20 user1 FAIL /page0
2026-10-05 10:30 user4 FAIL /page2
2026-10-05 10:40 user0 FAIL /page0
2026-10-05 10:50 user3 FAIL /page2
2026-10-05 10:00 user6 FAIL /page0
2026-10-05 10:10 user2 FAIL /page2
2026-10-05 10:20 user5 FAIL /page0
2026-10-05 10:30 user1 FAIL /page2
2026-10-05 10:40 user4 FAIL /page0
2026-10-05 10:50 user0 FAIL /page2
2026-10-05 10:00 user3 FAIL /page0
2026-10-05 10:10 user6 FAIL /page2
2026-10-05 10:20 user2 FAIL /page0
2026-10-05 10:30 user5 FAIL /page2
2026-10-05 10:40 user1 FAIL /page0
2026-10-05 10:50 user4 FAIL /page2
2026-10-05 10:00 user0 FAIL /page0
2026-10-05 10:10 user3 FAIL /page2
2026-10-05 10:20 user6 FAIL /page0
2026-10-05 10:30 user2 FAIL /page2
2026-10-05 10:40 user5 FAIL /page0
2026-10-05 10:50 user1 FAIL /page2
2026-10-05 10:00 user4 FAIL /page0
2026-10-05 10:10 user0 FAIL /page2
2026-10-05 10:20 user3 FAIL /page0
grep: access.log
: No such file or directory
```

Figure 4.9 – Full run of oneliners.sh

5 YOUR FIRST SCRIPT

5.1 report.sh

The TODO is completed with a loop over `user0` to `user6`. `grep -w` matches the user as a whole word, so the pattern for `user1` cannot match a longer name such as `user10`.

```
dev1@dev1:~/os-lab3$ cat report.sh
#!/bin/bash
# usage: ./report.sh access.log
# Laboratory 3 - Part 5 Log Summary Script

log=$1
if [ ! -f "$log" ]; then
  echo "no such file: $log" >&2
  exit 1
fi

total=$(wc -l < "$log")
fails=$(grep -c FAIL "$log")
echo "$total requests, $fails failed"

# Completed TODO: iterate over user0 through user6 and report individual failure totals
for u in {0..6}; do
  u_fails=$(grep -w "user$u" "$log" | grep -c FAIL)
  echo "user$u: $u_fails failures"
done
```

Figure 5.1 – Contents of report.sh

```
dev1@dev1:~/os-lab3$ chmod -x report.sh ; ./report.sh access.log ; echo "exit code: $?"
bash: ./report.sh: Permission denied
exit code: 126
dev1@dev1:~/os-lab3$ chmod +x report.sh
dev1@dev1:~/os-lab3$ report.sh access.log
bash: report.sh: command not found
dev1@dev1:~/os-lab3$ ./report.sh access.log
500 requests, 100 failed
user0: 14 failures
user1: 14 failures
user2: 14 failures
user3: 15 failures
user4: 14 failures
user5: 15 failures
user6: 14 failures
```

Figure 5.2 – Running the script: without execute permission, without ./, and on access.log

```
dev1@dev1:~/os-lab3$ ./report.sh nope.log ; echo "exit code: $?"
no such file: nope.log
exit code: 1
dev1@dev1:~/os-lab3$ ./report.sh nope.log 2>/dev/null ; echo "exit code: $?"
exit code: 1
```

Figure 5.3 – Error handling for a missing file

5.2 Answers to the Observe questions

a) **What does #!/bin/bash do, and what happens before chmod +x?** The shebang tells the kernel, inside `execve()`, which interpreter runs the file: the kernel actually executes `/bin/bash ./report.sh access.log`. Without the execute bit the kernel refuses: `Permission denied`, exit code 126 (Figure 5.2). `bash report.sh` would still work, because then bash only reads the file.

b) **Why ./report.sh and not report.sh?** A name without a slash is looked up only in the PATH directories, and the current directory is not one of them, so `report.sh` gives `command not found` (Figure 5.2). Keeping `.` out of PATH prevents a file planted in some directory from shadowing a real command such as `ls`. `./` is an explicit path, so no search happens.

c) **Why >&2 and exit 1?** The error message goes to stderr, so it never mixes with the data on stdout: with `2>/dev/null` the message disappears but the exit code is still 1 (Figure 5.3). `exit 1` reports failure to the caller, so `./report.sh x.log && next` does not continue and cron or other scripts can detect the error.

5.3 Bonus: running report.sh with cron

The line below was added with `crontab`. It uses absolute paths because cron starts jobs in the home directory with a minimal PATH, and `>> ... 2>&1` appends both output streams to the log. After two and a half minutes the log held the complete report from each minute. The line was then removed, leaving the existing crontab entries untouched (Figure 5.4).

```
* * * * * /home/dev1/os-lab3/report.sh /home/dev1/os-lab3/access.log
        >> /home/dev1/os-lab3/report.log 2>&1
```

```
dev1@dev1:~/os-lab3$ (crontab -l; echo '* * * * * /home/dev1/os-lab3/report.sh /home/dev1/os-lab3/access
.log >> /home/dev1/os-lab3/report.log 2>&1') | crontab -
dev1@dev1:~/os-lab3$ crontab -l | grep report.sh
* * * * * /home/dev1/os-lab3/report.sh /home/dev1/os-lab3/access.log >> /home/dev1/os-lab3/report.log 2>
&1
dev1@dev1:~/os-lab3$ sleep 150
dev1@dev1:~/os-lab3$ cat report.log
500 requests, 100 failed
user0: 14 failures
user1: 14 failures
user2: 14 failures
user3: 15 failures
user4: 14 failures
user5: 15 failures
user6: 14 failures
500 requests, 100 failed
user0: 14 failures
user1: 14 failures
user2: 14 failures
user3: 15 failures
user4: 14 failures
user5: 15 failures
user6: 14 failures
500 requests, 100 failed
user0: 14 failures
user1: 14 failures
user2: 14 failures
user3: 15 failures
user4: 14 failures
user5: 15 failures
user6: 14 failures
dev1@dev1:~/os-lab3$ crontab -l | grep -v os-lab3/report.sh | crontab -
dev1@dev1:~/os-lab3$ crontab -l | grep -c report.sh
0
```

Figure 5.4 – The crontab line, the log it produced, and its removal

CONCLUSIONS

The main lesson of this lab is that the shell, not the program, does most of the work on a command line. `echo` never saw `*.c`; it received ready file names. `ls alfa beta.txt` failed before `ls` had a chance to understand the name, and `wc` printed no file name because it never received one. Once the expansion order is clear (braces, variables and substitutions, then word splitting, then globbing, then quote removal), the output can be predicted. The only misprediction, `[a-b]*`, came from overlooking a file, not from the rules.

Redirection is just as mechanical: file descriptors are copied left to right, so `2>&1` has to come after the redirection it copies. The control operators `;`, `&&` and `||` depend only on exit codes, and the same codes made the script errors visible: 127 for a missing command, 126 for a missing execute bit, 1 for the script's own error.

The one-liners answered every question about the 500-line log with combinations of `grep`, `cut`, `sort`, `uniq` and `wc`. Each result was verified twice: by the handout's self-checks ($125 \times 4 = 500$, the per-user failures sum to 100) and by modular arithmetic on the generator (100, 72/15, 50). The Q4 solution prints every user tied at the maximum, so it stays correct when the log is regenerated with different data. Finally, `report.sh` turned the pipelines into a reusable program with argument checking, a correct error stream and exit code, and it ran unattended under cron.

REFERENCES

1. Free Software Foundation. *Bash Reference Manual*, version 5.2. Shell Expansions; Redirections.
2. IEEE & The Open Group. *POSIX.1-2017*, Shell Command Language.
3. Shotts, W. (2019). *The Linux Command Line* (2nd ed.). No Starch Press.
4. GNU coreutils manual: `ls` quoting styles.